

👉 Если есть пример, то его всегда показывает преподаватель в Jupyter! Студенты повторяют у себя на компьютерах

👉 Импортировать необходимые библиотеки: `import numpy as np`

# Revision. Numpy

## 1) Создание массивов в NumPy

Создание массивов - это фундаментальный аспект использования NumPy, поскольку массивы являются основной структурой данных, используемой для численных вычислений. Здесь мы рассмотрим, как создавать массивы с помощью `np.array()` и как создавать массивы из списков и кортежей.

Использование `np.array()`

Функция `np.array()` - это самый простой способ создания массива в NumPy. Она преобразует список, кортеж или другой объект, похожий на последовательность, в массив NumPy.

```
numpy.array(object, dtype=None, copy=True, order='K', ndmin=0)
```

- `object`: Входные данные, обычно список или кортеж.
- `dtype`: Желаемый тип данных массива (необязательно).
- `copy`: Если `True` (по умолчанию), объект копируется. Если `False`, копирование производится только в случае необходимости.
- `order`: Порядок расположения в памяти ('C' для row-major, 'F' для column-major, 'A' для any, 'K' для unchanged).
- `ndmin`: Указывает минимальное количество измерений, которое должен иметь результирующий массив.

Пример:

```
import numpy as np

# Создание одномерного массива из списка
list_data = [1, 2, 3, 4, 5]
array_from_list = np.array(list_data)
print("Массив из списка:", array_from_list)

# Создание одномерного массива из кортежа
tuple_data = (6, 7, 8, 9, 10)
array_from_tuple = np.array(tuple_data)
print("Массив из кортежа:", array_from_tuple)

# Создание двумерного массива из списка списков
```

```
list_of_lists = [[1, 2, 3], [4, 5, 6]]
array_2d = np.array(list_of_lists)
print("Двумерный массив из списка списков:\n", array_2d)
```

Выходные данные:

```
Массив из списка: [1 2 3 4 5]
Массив из кортежа: [ 6  7  8  9 10]
Двумерный массив из списка списков:
[[1 2 3]
 [4 5 6]]
```

## 2) Атрибуты массивов в NumPy

Массивы NumPy имеют несколько атрибутов, которые предоставляют ценную информацию о структуре и данных массива. Понимание этих атрибутов очень важно для эффективного манипулирования данными и их анализа. Здесь мы рассмотрим некоторые из наиболее часто используемых атрибутов: shape, size, ndim, dtype и itemsize.

### 1. Атрибут ndim

Атрибут ndim возвращает количество измерений (осей) массива. Это полезно для понимания сложности структуры массива.

Пример:

```
import numpy as np

# Создание одномерного массива
array1 = np.array([1, 2, 3])
print(array1.ndim) # Вывод: 1

# Создание двумерного массива
array2 = np.array([[2, 3, 4], [5, 6, 7]])
print(array2.ndim) # Вывод: 2
```

### 2. Атрибут shape

Атрибут shape возвращает кортеж целых чисел, указывающих размер массива в каждом измерении. Это помогает понять расположение массива.

Пример:

```
import numpy as np

# Создание одномерного массива
```

```
array1 = np.array([1, 2, 3])
print(array1.shape) # Вывод: (3,)

# Создание двумерного массива
array2 = np.array([[3, 4, 5], [6, 7, 8]])
print(array2.shape) # Вывод: (2, 3)
```

### 3. Атрибут size

Атрибут size возвращает общее количество элементов в массиве. Это произведение размеров всех измерений.

Пример:

```
import numpy as np

# Создание одномерного массива
array1 = np.array([1, 2, 3])
print(array1.size) # Выходные данные: 3

# Создание двумерного массива
array2 = np.array([[3, 5, 7], [8, 9, 10]])
print(array2.size) # Вывод: 6
```

### 4. Атрибут dtype

Атрибут dtype возвращает тип данных элементов массива. Массивы NumPy однородны, то есть все элементы имеют одинаковый тип данных.

Пример:

```
import numpy as np

# Создание массива целых чисел
array1 = np.array([10, 20, 30])
print(array1.dtype) # Выходные данные: int64

# Создание массива строк
array2 = np.array(["Python", "NumPy", "Data"])
print(array2.dtype) # Вывод: <U6
```

### 5. Атрибут itemsize

Атрибут itemsize возвращает размер (в байтах) каждого элемента в массиве. Это полезно для понимания потребления памяти массивом.

Пример:

```
import numpy as np
```

```
# Создание массива 64-битных целых чисел
array1 = np.array([1, 2, 3], dtype=np.int64)
print(array1.itemsize) # Выходные данные: 8

# Создание массива 32-битных целых чисел
array2 = np.array([1, 2, 3], dtype=np.int32)
print(array2.itemsize) # Вывод: 4
```

Здесь массив1 состоит из элементов, занимающих по 8 байт, а массив2 - из элементов, занимающих по 4 байта.

Эти атрибуты предоставляют важную информацию о структуре и данных массивов NumPy, позволяя эффективно манипулировать данными и анализировать их.

### 3) Инициализация массивов в NumPy

NumPy предоставляет несколько функций для инициализации массивов определенными значениями или шаблонами. Эти функции полезны для создания массивов с заранее заданными значениями, которые затем можно использовать для различных вычислительных задач. Здесь мы рассмотрим четыре часто используемые функции для инициализации массивов: `np.zeros()`, `np.ones()`, `np.full()`.

#### 1. `np.zeros()`

Функция `np.zeros()` создает массив, заполненный нулями. Это полезно, когда вам нужен массив определенной формы, инициализированный нулем.

```
numpy.zeros(shape, dtype=float, order='C')
```

- `shape`: Форма массива (кортеж целых чисел).
- `dtype`: Желаемый тип данных массива (необязательно, по умолчанию `float`).
- `order`: Порядок расположения в памяти ("C" - основной по строкам, "F" - основной по столбцам).

Пример:

```
import numpy as np

# Создание одномерного массива нулей
array1 = np.zeros(5)
print("1D массив нулей:", array1)

# Создание двумерного массива нулей
```

```
array2 = np.zeros((3, 4))
print("Двумерный массив нулей:\n", array2)
```

Выходные данные:

```
1D массив нулей: [0. 0. 0. 0. 0.]
Двумерный массив нулей:
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
```

## 2. np.ones()

Функция `np.ones()` создает массив, заполненный единицами. Это полезно, когда вам нужен массив определенной формы, инициализированный единицей.

```
numpy.ones(shape, dtype=None, order='C')
```

- `shape`: Форма массива (кортеж целых чисел).
- `dtype`: Желаемый тип данных массива (необязательно, по умолчанию `float`).
- `order`: Порядок расположения в памяти ("C" - основной по строкам, "F" - основной по столбцам).

Пример:

```
import numpy as np

# Создание одномерного массива единиц
array1 = np.ones(5)
print("одномерный массив единиц:", array1)

# Создание двумерного массива единиц
array2 = np.ones((3, 4))
print("Двумерный массив единиц:\n", array2)
```

Выходные данные:

```
одномерный массив единиц: [1. 1. 1. 1. 1.]
Двумерный массив единиц:
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

## 3. np.full()

Функция `np.full()` создает массив, заполненный заданным значением. Это полезно, когда вам нужен массив определенной формы, инициализированный определенным значением.

```
python
numpy.full(shape, fill_value, dtype=None, orde
```

- `shape`: Форма массива (кортеж целых чисел).
- `fill_value`: Значение, которым заполняется массив.
- `dtype`: Желаемый тип данных массива (необязательно).
- `order`: Порядок расположения в памяти ("C" - основной по строкам, "F" - основной по столбцам).

Пример:

```
import numpy as np

# Создание одномерного массива, заполненного значением 7
array1 = np.full(5, 7)
print("Одномерный массив, заполненный значением 7:", array1)

# Создание двумерного массива, заполненного значением 3.14
array2 = np.full((3, 4), 3.14)
print("Двумерный массив, заполненный значением 3.14:\n", array2)
```

Выходные данные:

```
Одномерный массив, заполненный значением 7: [7 7 7 7 7]
Двумерный массив, заполненный значением 3.14:
[[3.14 3.14 3.14 3.14]
 [3.14 3.14 3.14 3.14]
 [3.14 3.14 3.14 3.14]]
```

Эти функции обеспечивают удобный способ инициализации массивов определенными значениями, которые затем могут быть использованы для различных вычислительных задач в анализе данных и научных вычислениях.

#### 4) Инициализация массивов: Создание последовательностей с помощью `np.arange()` и `np.linspace()`

NumPy предоставляет функции для создания массивов с последовательностями чисел. Две наиболее часто используемые для этого функции - `np.arange()` и `np.linspace()`. Эти функции позволяют создавать массивы с равномерно

распределенными значениями, что может быть очень полезно в различных численных задачах и задачах анализа данных.

## 1. np.arange()

Функция `np.arange()` генерирует массив с равномерно распределенными значениями в заданном диапазоне. Она похожа на встроенную в Python функцию `range()`, но возвращает не список, а массив NumPy.

```
numpy.arange([start, ]stop, [step, ]dtype=None)
```

- `start`: Начальное значение последовательности (включительно). По умолчанию равно 0.
- `stop`: конечное значение последовательности (включительно).
- `step` (шаг): Расстояние между значениями. По умолчанию равно 1.
- `dtype` (тип): Желаемый тип данных массива (необязательно).

Пример:

```
import numpy as np

# Создание массива со значениями от 0 до 9
array1 = np.arange(10)
print("Массив со значениями от 0 до 9:", array1)

# Создание массива со значениями от 1 до 9
array2 = np.arange(1, 10)
print("Массив со значениями от 1 до 9:", array2)

# Создание массива со значениями от 1 до 9 с шагом 2
array3 = np.arange(1, 10, 2)
print("Массив со значениями от 1 до 9 с шагом 2:", array3)

# Создание массива со значениями с плавающей точкой
array4 = np.arange(0, 1, 0.1)
print("Массив со значениями с плавающей точкой от 0 до 1 с шагом 0,1:", array4)
```

Выходные данные:

```
Массив со значениями от 0 до 9: [0 1 2 3 4 5 6 7 8 9]
Массив со значениями от 1 до 9: [1 2 3 4 5 6 7 8 9]
Массив со значениями от 1 до 9 с шагом 2: [1 3 5 7 9]
Массив со значениями с плавающей точкой от 0 до 1 с шагом 0,1: [0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
```

## 2. np.linspace()

Функция `np.linspace()` генерирует массив с заданным количеством равномерно распределенных значений в заданном интервале. Это особенно полезно, когда вам нужно определенное количество точек между двумя значениями.

Синтаксис:

```
numpy.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0)
```

- `start`: Начальное значение последовательности.
- `stop`: Конечное значение последовательности.
- `num`: Количество равномерно распределенных образцов для генерации. По умолчанию 50.
- `endpoint`: Если `True` (по умолчанию), то `stop` - это последний образец. Если `False`, то он не включается.
- `retstep`: Если `True`, возвращает размер шага между выборками.
- `dtype` (тип данных): Желаемый тип данных массива (необязательно).
- `axis` (ось): Ось в результате, вдоль которой хранятся образцы. По умолчанию 0.

Пример:

```
import numpy as np

# Создание массива с 10 равномерно распределенными значениями от 0 до 1
array1 = np.linspace(0, 1, 10)
print("Массив с 10 равномерно распределенными значениями от 0 до 1:", array1)

# Создание массива с 5 равномерно распределенными значениями от 1 до 10
array2 = np.linspace(1, 10, 5)
print("Массив с 5 равномерно распределенными значениями от 1 до 10:", array2)

# Создание массива с 5 равномерно распределенными значениями от 1 до 10,
исключая конечную точку
array3 = np.linspace(1, 10, 5, endpoint=False)
print("Массив с 5 равномерно распределенными значениями от 1 до 10, исключая
конечную точку:", array3)

# Создание массива и возвращение размера шага
array4, step = np.linspace(0, 1, 5, retstep=True)
print("Массив с 5 равномерно расположенными значениями от 0 до 1:", array4)
print("Размер шага:", step)
```

Выходные данные:



```
Массив с 10 равномерно распределенными значениями от 0 до 1: [0.        0.11111111 0.22222222 0.33333333 0.44444444 0.55555556
0.66666667 0.77777778 0.88888889 1.        ]
Массив с 5 равномерно распределенными значениями от 1 до 10: [ 1.   3.25  5.5   7.75 10.   ]
Массив с 5 равномерно распределенными значениями от 1 до 10, исключая конечную точку: [1.  2.8  4.6  6.4  8.2]
Массив с 5 равномерно расположенными значениями от 0 до 1: [0.   0.25 0.5   0.75 1.   ]
Размер шага: 0.25
```

Эти функции обеспечивают гибкие и эффективные способы создания массивов с последовательностями чисел, которые необходимы для решения различных задач численного анализа и анализа данных.

## 5) Арифметические операции в NumPy: Поэлементные операции и broadcasting

NumPy предоставляет широкий спектр арифметических операций, которые можно выполнять над массивами. Эти операции эффективны и могут применяться поэлементно, что делает NumPy предпочтительным выбором для работы с массивами. Кроме того, NumPy поддерживает broadcasting - технику, которая позволяет выполнять арифметические операции над массивами различной формы при определенных условиях.

### *Поэлементные операции*

Поэлементные операции в NumPy - это операции, которые применяются к каждому соответствующему элементу массивов. К таким операциям относятся сложение, вычитание, умножение, деление и другие. NumPy предоставляет как операторы, так и функции для выполнения этих операций.

Сложение:

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = a + b
print(result) # Вывод: [5 7 9]

result = np.add(a, b)
print(result) # Вывод: [5 7 9]
```

Вычитание:

```
result = a - b
print(result) # Вывод: [-3 -3 -3]

result = np.subtract(a, b)
print(result) # Вывод: [-3 -3 -3]
```

Умножение:

```
result = a * b
print(result) # Вывод: [ 4 10 18]

result = np.multiply(a, b)
print(result) # Вывод: [ 4 10 18]
```

Деление:

```
result = a / b
print(result) # Вывод: [0.25 0.4 0.5 ]

result = np.divide(a, b)
print(result) # Вывод: [0.25 0.4 0.5 ]
```

Возведение в степень:

```
result= a ** b
print(result) # Вывод: [ 1 32 729]

result = np.power(a, b)
print(result) # Вывод: [ 1 32 729]
```

### *Broadcasting*

Broadcasting - это мощная функция в NumPy, которая позволяет выполнять арифметические операции над массивами различной формы. При выполнении операций над массивами разного размера NumPy автоматически расширяет меньший массив, чтобы он соответствовал форме большего массива, следуя определенным правилам.

Правила broadcasting'a:

1. Если массивы не имеют одинакового количества измерений, добавляйте 1 к форме меньшего массива до тех пор, пока обе формы не станут одинаковой длины.
2. Массивы совместимы для broadcasting'a, если для каждой размерности их размеры либо одинаковы, либо один из них равен 1.
3. Результирующая форма массива - это максимальный размер по каждому измерению входных массивов.

Пример:

```
import numpy as np
```

```
# Массив a формы (3, 1)
a = np.array([[1], [2], [3]])

# Массив b формы (1, 4)
b = np.array([4, 5, 6, 7])

# Передача a и b в форму (3, 4)
result = a + b
print(result)
```

Выходные данные:

```
[[ 5  6  7  8]
 [ 6  7  8  9]
 [ 7  8  9 10]]
```

В этом примере массив a с формой (3, 1) транслируется для соответствия форме массива b с формой (1, 4). В результате получается массив формы (3, 4), каждый элемент которого представляет собой сумму соответствующих элементов из a и b.

- Поэлементные операции: Эти операции применяются к каждому элементу соответствующих массивов. NumPy предоставляет как операторы, так и функции для этих операций.
- Broadcasting: Эта функция позволяет выполнять арифметические операции над массивами разной формы, автоматически расширяя меньший массив в соответствии с формой большего массива, следуя определенным правилам.

## 6) Операции над матрицами

### Умножение матриц

Умножение матриц можно выполнить с помощью операторов `np.dot()`, `np.matmul()` или `@`. Количество столбцов в первой матрице должно быть равно количеству строк во второй матрице.

Пример:

```
import numpy as np

# Создание двух матриц
matrix1 = np.array([[1, 3], [5, 7]])
matrix2 = np.array([[2, 6], [4, 8]])

# Умножение матриц
```

```
result = np.dot(matrix1, matrix2)
print("Матричное умножение:\n", result)
```

Вывод:

```
Матричное умножение :
[[14 30]
 [38 86]]
```

### Транспонирование матрицы

Транспонирование матрицы получается, если поменять местами ее строки и столбцы. Это можно сделать с помощью функции `np.transpose()` или атрибута `.T`.

Пример:

```
import numpy as np

# Создание матрицы
matrix = np.array([[1, 3], [5, 7]])

# Транспонирование матрицы
transpose = np.transpose(matrix)
print("Транспонирование:\n", transpose)
```

Вывод:

```
Транспонирование :
[[1 5]
 [3 7]]
```

## 7) Изменение формы и размера массивов в NumPy

NumPy предоставляет функции для изменения формы и размера массивов, позволяющие гибко манипулировать их размерами. Эти операции необходимы для решения различных задач численного анализа и анализа данных. Здесь мы рассмотрим `np.reshape()`, `np.resize()`, а также методы добавления и удаления размеров с помощью `np.newaxis` и `np.expand_dims()`.

### Изменение формы массивов: `np.array.reshape()`

Метод `np.array.reshape()` изменяет форму массива, не изменяя его данных. Общее количество элементов должно оставаться неизменным.

```
numpy.array.reshape(newshape)
```

- newshape: Новая форма (кортеж целых чисел).

Примеры:

```
import numpy as np

# 1D в 2D
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
print("От 1D к 2D:\n", newarr)

# 1D в 3D
newarr = arr.reshape(2, 3, 2)
print("От 1D к 3D:\n", newarr)

# Использование -1 для неизвестной размерности
newarr = arr.reshape(2, 2, -1)
print("Использование -1 для неизвестной размерности:\n", newarr)
```

Вывод:

```
От 1D к 2D:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
От 1D к 3D:
[[[ 1  2]
 [ 3  4]
 [ 5  6]]

 [[ 7  8]
 [ 9 10]
 [11 12]]]
Использование -1 для неизвестной размерности:
[[[ 1  2  3]
 [ 4  5  6]]

 [[ 7  8  9]
 [10 11 12]]]
```

Изменение размера массивов: `np.resize()`

Функция `np.resize()` изменяет форму массива и может изменять общее количество элементов. Если новый размер больше, массив заполняется повторяющимися копиями исходных данных.

```
numpy.resize(a, new_shape)
```

- `a`: Массив, размер которого нужно изменить.
- `new_shape`: Новая форма (кортеж целых чисел).

Примеры:

```
import numpy as np

# Исходный массив
arr = np.array([[0, 1], [2, 3]])

# Изменение размера массива в большую сторону
resized_arr = np.resize(arr, (2, 3))
print("Изменен размер массива на больший:\n", resized_arr)

# Изменение размера массива в меньшую сторону
resized_arr = np.resize(arr, (1, 4))
print("Изменен размер массива на меньший:\n", resized_arr)
```

Вывод:

```
Изменен размер массива на больший:
[[0 1 2]
 [3 0 1]]
Изменен размер массива на меньший:
[[0 1 2 3]]
```

*Добавление размеров: `np.newaxis`*

`np.newaxis`: Используется для увеличения размерности массива еще на одно измерение, делая его более высокоразмерным.

Пример:

```
import numpy as np

# Исходный одномерный массив
arr = np.array([1, 2, 3])

# Добавление новой оси
newarr = arr[:, np.newaxis]
```

```
print("Добавление новой оси с помощью np.newaxis:\n", newarr)
```

Вывод:

```
Добавление новой оси с помощью np.newaxis:  
[[1]  
 [2]  
 [3]]
```

## 8) Случайные числа в NumPy

*Генерация случайных чисел*

1. np.random.rand()

Функция np.random.rand() генерирует случайные числа из равномерного распределения на интервале [0, 1). Она может генерировать случайные числа любой формы, заданной аргументами.

```
numpy.random.rand(d0, d1, ..., dn)
```

- d0, d1, ..., dn: Размеры возвращаемого массива.

Пример:

```
import numpy as np  
  
# Генерация одного случайного числа  
random_number = np.random.rand()  
print("Одиночное случайное число:", random_number)  
  
# Генерация одномерного массива случайных чисел  
random_array_1d = np.random.rand(5)  
print("Одномерный массив случайных чисел:", random_array_1d)  
  
# Генерация двумерного массива случайных чисел  
random_array_2d = np.random.rand(3, 4)  
print("Двумерный массив случайных чисел:\n", random_array_2d)
```

Выходные данные:

```
Одиночное случайное число: 0.3439993495686171  
Одномерный массив случайных чисел: [0.51118773 0.49025769 0.98302143 0.73244607 0.00378275]  
Двумерный массив случайных чисел:  
[[0.48992219 0.9731271 0.28731655 0.7431044 ]  
 [0.01041924 0.29597925 0.57869845 0.29415558]  
 [0.1650352 0.06879116 0.12179249 0.61111302]]
```

## 2. np.random.randn()

Функция `np.random.randn()` генерирует случайные числа из стандартного нормального распределения (среднее = 0, стандартное отклонение = 1). Она может генерировать случайные числа любой формы, заданной аргументами.

```
numpy.random.randn(d0, d1, ..., dn)
```

- `d0, d1, ..., dn`: Размеры возвращаемого массива.

Пример:

```
import numpy as np

# Генерация одного случайного числа из стандартного нормального
# распределения
random_number = np.random.randn()
print("Одиночное случайное число из стандартного нормального распределения:",
      random_number)

# Генерация одномерного массива случайных чисел из стандартного нормального
# распределения
random_array_1d = np.random.randn(5)
print("1D массив случайных чисел из стандартного нормального распределения:",
      random_array_1d)

# Генерирование двумерного массива случайных чисел из стандартного
# нормального распределения
random_array_2d = np.random.randn(3, 4)
print("Двумерный массив случайных чисел из стандартного нормального
распределения:\n", random_array_2d)
```

Выходные данные:

```
Одиночное случайное число из стандартного нормального распределения: -0.6687640529109288
1D массив случайных чисел из стандартного нормального распределения: [ 0.79550431 -0.18718504  0.63239159 -0.95170626  0.9031778 ]
Двумерный массив случайных чисел из стандартного нормального распределения:
[[-0.99511721 -0.26859096 -1.88186055 -1.22788821]
 [-0.2010815  -0.49854496  0.62376896  1.27257252]
 [ 0.49228158 -0.02089429  1.73461492  0.34107269]]
```

## 3. np.random.randint()

Функция `np.random.randint()` генерирует случайные целые числа из заданного диапазона. Она может генерировать случайные целые числа любой формы, заданной аргументами.

```
numpy.random.randint(low, high=None, size=None, dtype=int)
```



- low: Наименьшее целое число (включительно).
- high: Наибольшее целое число (включительно). Если None, то диапазон будет [0, low).
- size: Форма возвращаемого массива (необязательно).
- dtype: Желаемый тип данных массива (необязательно, по умолчанию int).

Пример:

```
import numpy as np

# Генерация одного случайного целого числа
random_int = np.random.randint(10)
print("Одиночное случайное целое число:", random_int)

# Генерация одномерного массива случайных целых чисел
random_array_1d = np.random.randint(1, 10, 5)
print("1D массив случайных целых чисел:", random_array_1d)

# Генерирование двумерного массива случайных целых чисел
random_array_2d = np.random.randint(1, 10, (3, 4))
print("Двумерный массив случайных целых чисел:\n", random_array_2d)
```

Выходные данные:

```
Одиночное случайное целое число: 1
1D массив случайных целых чисел: [7 4 3 4 7]
Двумерный массив случайных целых чисел:
[[9 9 5 1]
 [7 9 9 8]
 [3 6 2 6]]
```

Случайная выборка: `np.random.choice()`

Функция `np.random.choice()` генерирует случайную выборку из заданного одномерного массива. Ее можно использовать для случайного выбора элементов из массива с заменой или без нее.

```
numpy.random.choice(a, size=None, replace=True, p=None)
```

- a: Входной массив или диапазон.
- size: Форма возвращаемого массива (необязательно).
- replace: Выборка с заменой или без замены (по умолчанию True).
- p: Вероятности, связанные с каждой записью в a (необязательно).

Пример:

```
import numpy as np

# Создание массива
arr = np.array([1, 2, 3, 4, 5])

# Случайный выбор одного элемента
random_choice = np.random.choice(arr)
print("Случайно выбранный элемент:", random_choice)

# Случайный выбор нескольких элементов с заменой
random_choices = np.random.choice(arr, size=3, replace=True)
print("Случайно выбранные элементы с заменой:", random_choices)

# Случайный выбор нескольких элементов без замены
random_choices = np.random.choice(arr, size=3, replace=False)
print("Случайный выбор элементов без замены:", random_choices)

# Случайный выбор элементов с заданными вероятностями
random_choices = np.random.choice(arr, size=3, p=[0.1, 0.2, 0.3, 0.2, 0.2])
print("Случайно выбранные элементы с заданными вероятностями:",
      random_choices)
```

Вывод:

```
Случайно выбранный элемент: 1
Случайно выбранные элементы с заменой: [1 1 4]
Случайный выбор элементов без замены: [3 1 2]
Случайно выбранные элементы с заданными вероятностями: [5 2 2]
```

Воспроизводимость: `np.random.seed()`

Функция `np.random.seed()` задает параметр для генератора случайных чисел NumPy. Установка `seed` обеспечивает воспроизводимость генерируемых случайных чисел, что важно для отладки и обмена результатами.

```
numpy.random.seed(seed)
```

- `seed`: Значение параметра (целое число).

Пример:

```
import numpy as np

# Установка семени
```

```

np.random.seed(42)

# Генерация случайных чисел
random_array_1 = np.random.rand(3)
print("Случайный массив 1:", random_array_1)

# Сброс затравки
np.random.seed(42)

# Снова генерируем случайные числа
random_array_2 = np.random.rand(3)
print("Случайный массив 2:", random_array_2)

```

Выходные данные:

```

Случайный массив 1: [0.37454012 0.95071431 0.73199394]
Случайный массив 2: [0.37454012 0.95071431 0.73199394]

```

## Решение задач

- 1) Создайте одномерный массив, преобразуйте его в двумерный, а затем "сплющите" обратно в одномерный.

```

import numpy as np

array_1d = np.arange(1, 13)
print("1D array:", array_1d)

array_2d = array_1d.reshape(3, 4)
print("2D array:\n", array_2d)

flattened_array = array_2d.flatten()
print("Flattened array:", flattened_array)

```

- 2) Сгенерируйте случайный массив и рассчитайте базовую статистику.

```

import numpy as np

np.random.seed(42)

random_array = np.random.rand(4, 5)
print("Random array:\n", random_array)

```

```
mean_value = np.mean(random_array)
std_deviation = np.std(random_array)
min_value = np.min(random_array)
max_value = np.max(random_array)

print("Mean value:", mean_value)
print("Standard deviation:", std_deviation)
print("Minimum value:", min_value)
print("Maximum value:", max_value)
```

## Вопросы для закрепления

- 1) Какие операторы позволяют умножать матрицы друг на друга?
- 2) Какую функцию необходимо использовать для воспроизводимости результатов, связанных с генерацией случайных чисел?